AI ASSISTED CODING

Assignment-13.5

Name: A.sai shrehan

HTno:2303A52299

Batch:45

Task Description #1 (Refactoring – Removing Global Variables)

• Task: Use AI to eliminate unnecessary global variables from the code.

• Instructions:

o Identify global variables used across functions.

o Refactor the code to pass values using function parameters.

o Improve modularity and testability.

• Sample Legacy Code:

```
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
```

• Expected Output:

o Refactored version passing rate as a parameter or using a configuration structure.

```python
'''Task Description #1 (Refactoring Removing Global Variables)
• Task: Use AI to eliminate unnecessary global variables from the
code.
• Instructions:
o Identify global variables used across functions.
o Refactor the code to pass values using function parameters.
o Improve modularity and testability.
• Sample Legacy Code:
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
• Expected Output:
o Refactored version passing rate as a parameter or using a
configuration structure.'''

# Refactored version passing rate as a parameter or using a configuration structure.
def calculate_interest(amount, rate):
    return amount * rate
print(calculate_interest(1000, 0.1))
```

```
PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/.venv/Scripts/python.exe c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/assignment_13-5/task1.py
100.0
PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:\Users\Sushri\Documents\3-2\AI-Assisted_coding\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding>
```

Task Description #2 : (Refactoring Deeply Nested Conditionals)

• Task: Use AI to refactor deeply nested if–elif–else logic into a

cleaner structure.

• Focus Areas:

o Readability

o Logical simplification

o Maintainability

Legacy Code:

score = 78

if score >= 90:

print("Excellent")

else:

if score >= 75:

print("Very Good")

else:

if score >= 60:

print("Good")

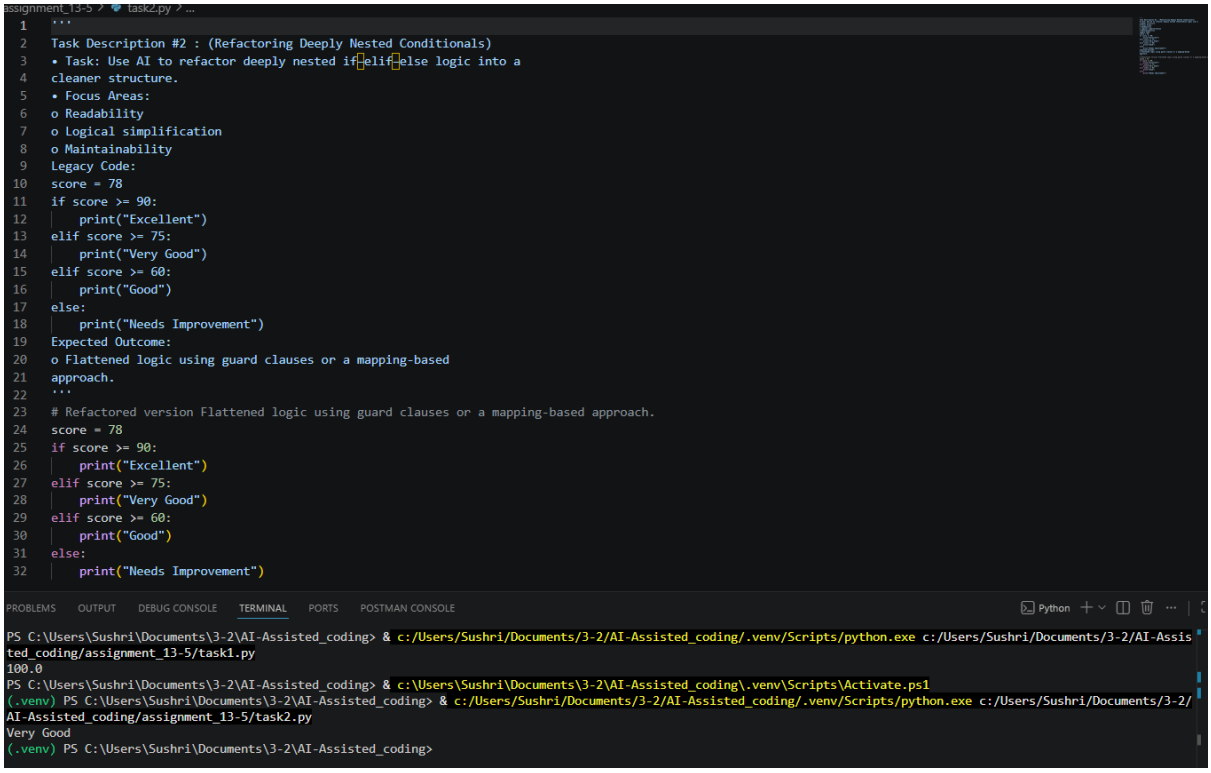
else:

print("Needs Improvement")

Expected Outcome:

o Flattened logic using guard clauses or a mapping-based

approach.



```
assignment_13-5 > task2.py > ...
'''
Task Description #2 : (Refactoring Deeply Nested Conditionals)
• Task: Use AI to refactor deeply nested if/elif/else logic into a
cleaner structure.
• Focus Areas:
o Readability
o Logical simplification
o Maintainability
Legacy Code:
score = 78
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:
    print("Needs Improvement")
Expected Outcome:
o Flattened logic using guard clauses or a mapping-based
approach.
'''
# Refactored version Flattened logic using guard clauses or a mapping-based approach.
score = 78
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:
    print("Needs Improvement")
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE        Python
PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:/Users/Sushri/Documents/3-2/AI-Assis
ted_coding/assignment_13-5/task1.py
100.0
PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:\Users\Sushri\Documents\3-2\AI-Assisted_coding\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:/Users/Sushri/Documents/3-2/
AI-Assisted_coding/assignment_13-5/task2.py
Very Good
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding>
```

Task 3 (Refactoring Repeated File Handling Code)

• Task: Use AI to refactor repeated file open/read/close logic.

• Focus Areas:

o DRY principle

o Context managers

o Function reuse

Legacy Code:

f = open("data1.txt")

print(f.read())

f.close()

f = open("data2.txt")

print(f.read())

f.close()

Expected Outcome:

o Reusable function using with open() and parameters.

```
assignment_13-5 > task3.py > ...
Task 3 (Refactoring Repeated File Handling Code)
• Task: Use AI to refactor repeated file open/read/close logic.
• Focus Areas:
o DRY principle
o Context managers
o Function reuse
Legacy Code:
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
f.close()
Expected Outcome:
o Reusable function using with open() and parameters.
'''
# Refactored version using a reusable function with context managers.
import os

def read_file(filename):
    with open(filename) as f:
        print(f.read())

# Get the directory of the current script
script_dir = os.path.dirname(os.path.abspath(__file__))
read_file(os.path.join(script_dir, "data1.txt"))
read_file(os.path.join(script_dir, "data2.txt"))
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/.venv/Scripts/py
thon.exe c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/assignment_13-5/task3.py
    read_file(os.path.join(script_dir, "data1.txt"))
  File "c:\Users\Sushri\Documents\3-2\AI-Assisted_coding\assignment_13-5\task3.py", line 22, in read_file
    with open(filename) as f:
         ^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'c:\\Users\\Sushri\\Documents\\3-2\\AI-Assisted_coding\\assignment_13-5\
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/.venv/Scripts/py
thon.exe c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/assignment_13-5/task3.py
This is the content of data1.txt.
This is the content of data2.txt.
```

Task 4 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate

data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

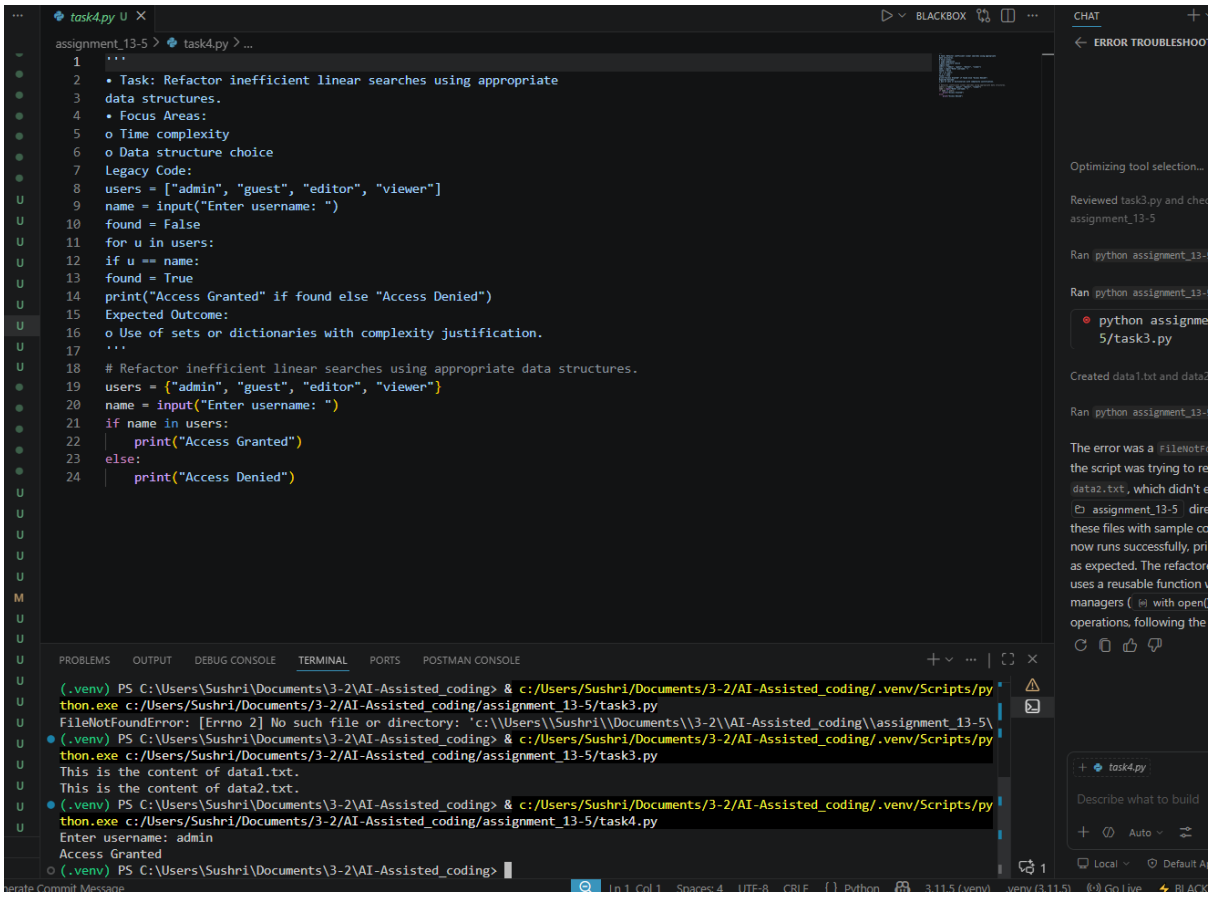
if u == name:

found = True

print("Access Granted" if found else "Access Denied")

Expected Outcome:

o Use of sets or dictionaries with complexity justification.



Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based

design.

• Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

salary = 50000

tax = salary * 0.2

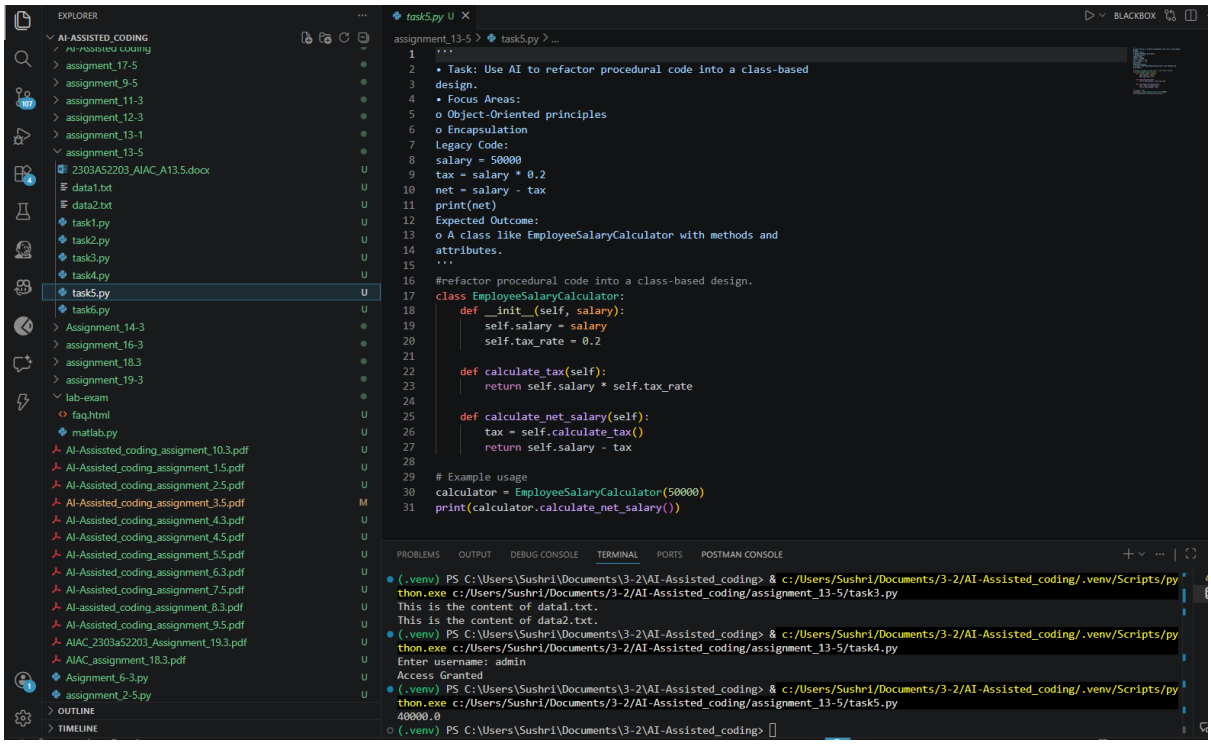
net = salary - tax

print(net)

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and

attributes.



Task 6 (Refactoring for Performance Optimization)

• Task: Use AI to refactor a performance-heavy loop handling

large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

total = 0

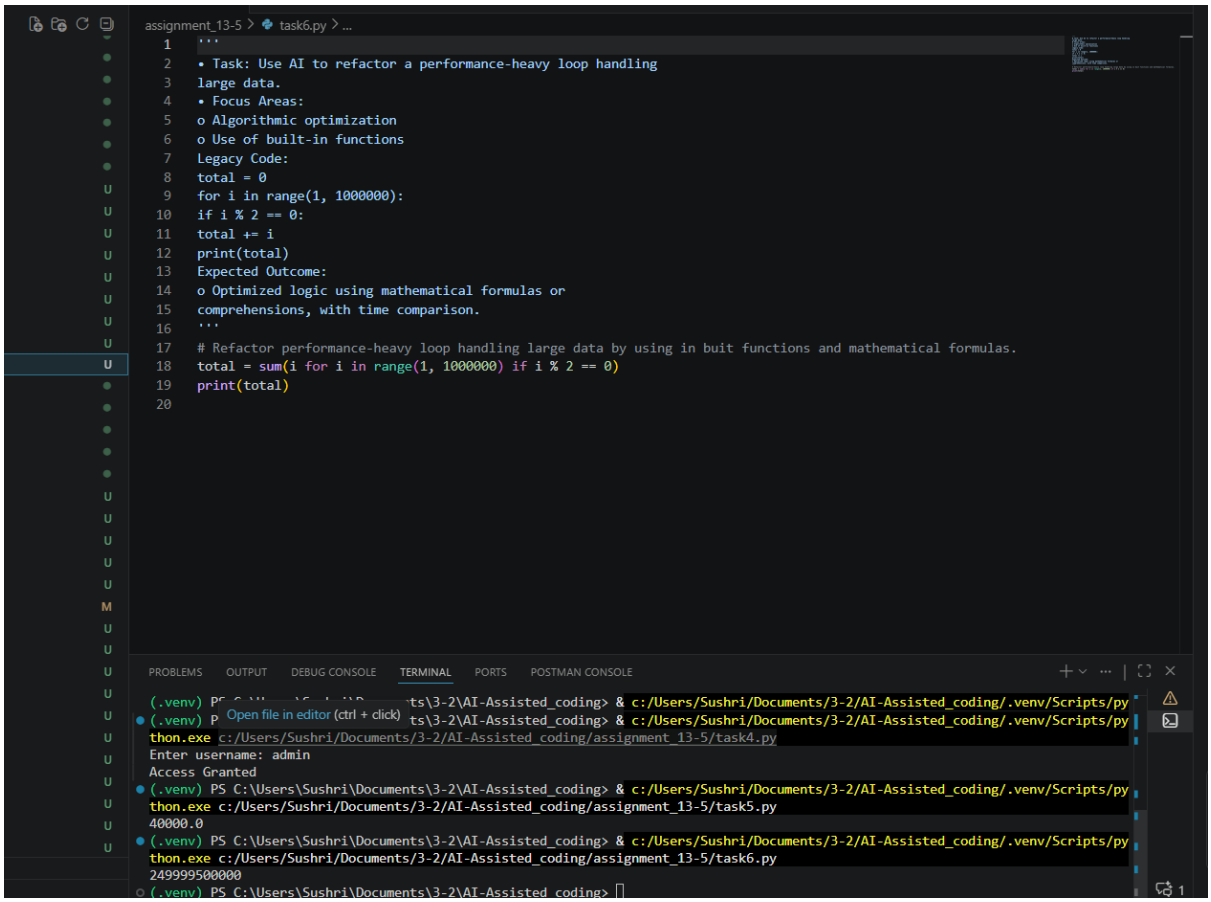
for i in range(1, 1000000):

if i % 2 == 0:

total += i

print(total)

Expected Outcome:

o Optimized logic using mathematical formulas or

comprehensions, with time comparison.